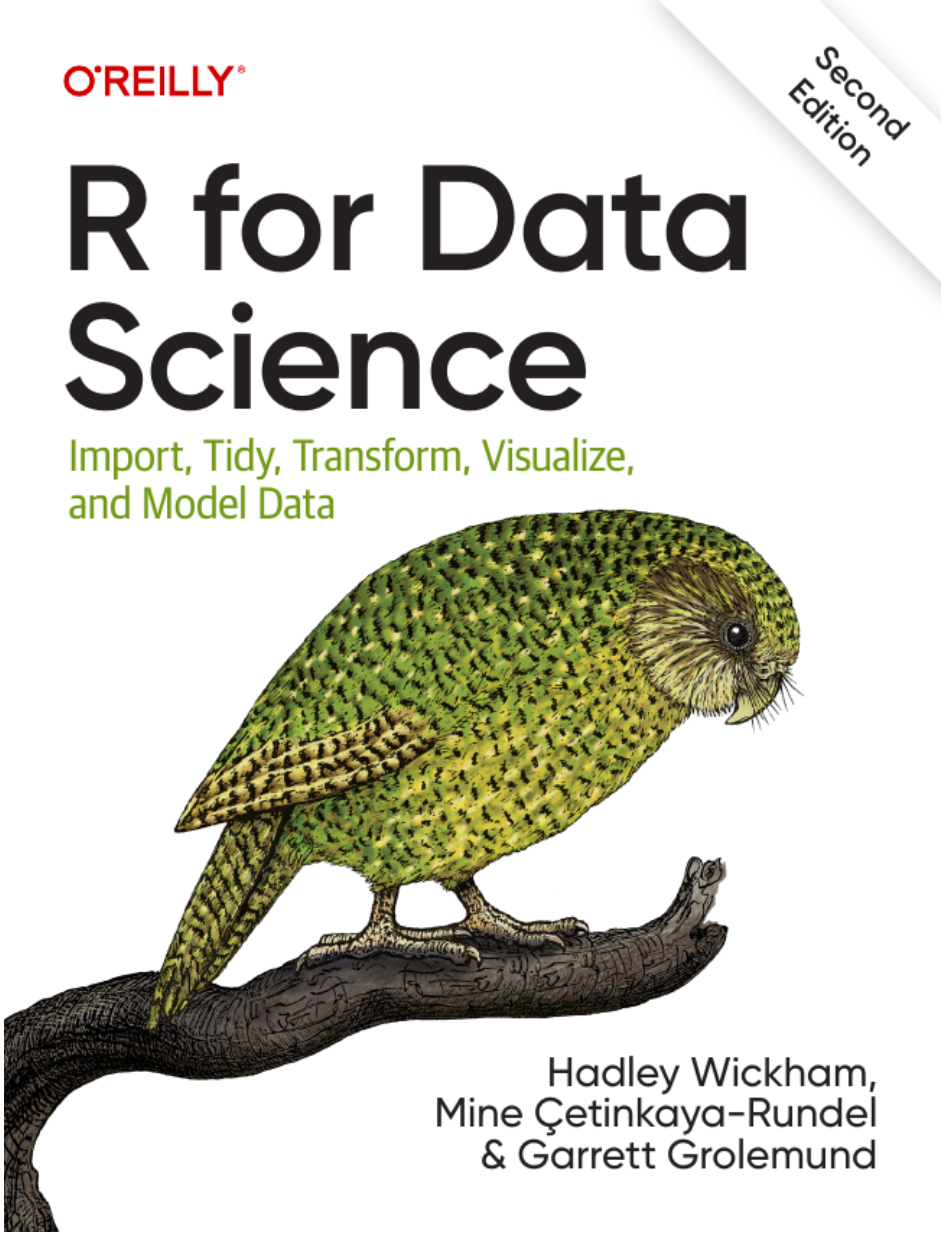
Scientific Programming
Chapter 5 - R and Statistical Testing

Jean-Sébastien Lerat

2023-2024

Main Reference



Introduction

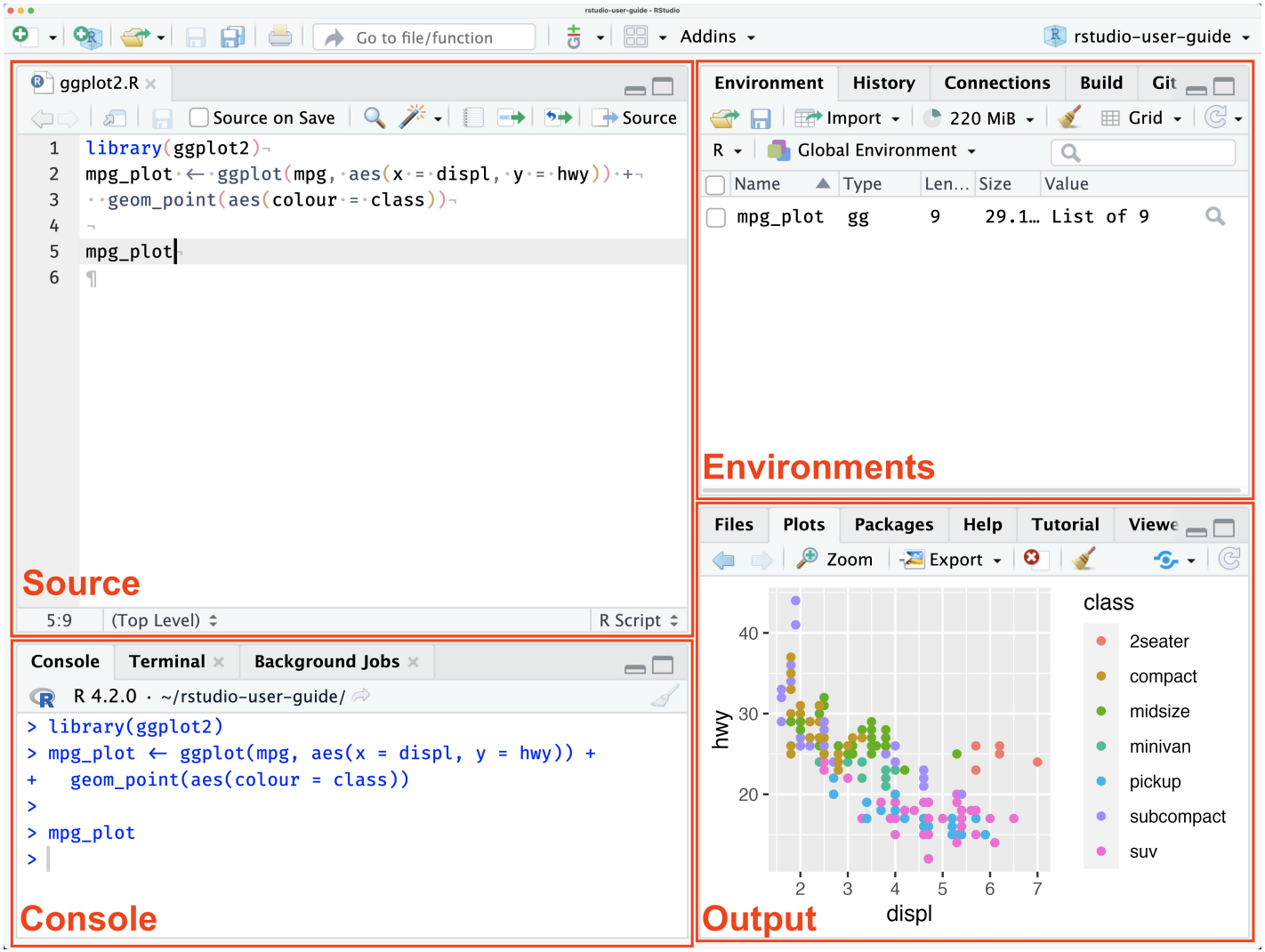
From the new side (`Julia`)

Introduction

To the old side (R)

Introduction

R and RStudio `https://posit.co/download/rstudio-desktop/`



Content

1. Introduction
2. R
   - Types and Variables
   - Operators
   - Vector and Matrix
   - Control Flow
   - Function
3. Describing Data
4. Distribution
   - Continuous
   - Discrete
   - Vizualisation
5. Testing Hypotheses
   - Adequacy
   - Conformity
6. Exercise

R – Types and Variables

Objects and Pirmitive Types:

```r
# Only single line comments

myInteger   <- 42L     # Integer require L for long
myComplex   <- 7 + 5i  # Complex
myNumber    <- 3.14    # Numeric (all other format)
myLogical   <- TRUE    # Logicial is either TRUE or FALSE
myCharacter <- "str"   # Character is 'chr' or "chr"

print(class(myNumber))          # Will print: numeric
print(typeof(myNumber))         # Will print: double
print(is.integer(myLogicial))   # Will print: FALSE
print(as.numeric(myInteger))    # Will print: 42

print(ls())      # Wil print all defined variables
rm(myInteger)    # Will destroy the variable myInteger
```

Valid identifiers ($\neq$ keywords) follow the regular expression:

$$\left(\texttt{[a-zA-Z.]|([.][a-zA-Z_])}\right)\texttt{[a-zA-Z0-9._]*}$$

R – Types and Variables

Affectation

```
a   _ 1L # The old-way, removed from specification
b  := 2L # A non-standard, old-way
c  <- 3L # The correct, recommanded way (leftward)
4L -> d  # The correct, recommanded way (rightward)
e   = 5L # The compatibility way
#          (recommended only as default parameters (functions))

# Crazy situation
f <- 42L -> g # f and g contains 42
```

R – Operators

Arithmetic Operators

```
1 + 2  # 3
2 - 2  # 0
3 * 2  # 6
4 / 2  # 2 (numeric)
5 ^ 2  # 25
6 %% 2 # 6 mod 2 = 0
# % is reserved to define ab operator or for special operations
#(ex: somthing contained in another object,
#     multiply a matrix with its transpose)


a <- c(1,2,3) # a vector
b <- c(4,5,6) # a vector

a+b  # 5 7 9
a-b  # -3 -3 -3
a*b  #  4 10 18
a/b  # 0.25 0.40 0.50
a^b  #   1  32 729
a%%b # 1 2 3
```

R – Operators

Relational Operators

```
1 < 2    # TRUE
2 > 2    # FALSE
3 <= 2   # FALSE
4 >= 2   # TRUE
5 == 2   # FALSE
6 != 2   # TRUE


a <- c(1,2,3) # a vector
b <- c(4,5,6) # a vector

a < b    # TRUE TRUE TRUE
a > b    # FALSE FALSE FALSE
a <= b   # TRUE TRUE TRUE
a >= b   # FALSE FALSE FALSE
a == b   # FALSE FALSE FALSE
a != b   # TRUE TRUE TRUE
```

R – Operators

Logical Operators

```
TRUE & FALSE        # Element-wise logical AND => FALSE
TRUE | FALSE        # Element-wise logical OR => TRUE
TRUE && FALSE       # Logical AND => FALSE
TRUE || FALSE       # Logical OR => TRUE
!TRUE               # Negation => FALSE

a <- c(TRUE,FALSE,TRUE,FALSE) # a vector
b <- c(TRUE,TRUE,FALSE,FALSE) # a vector

a & b    # TRUE FALSE FALSE FALSE
a && b   # FALSE (no more supported)

a | b    # TRUE  TRUE  TRUE FALSE
a || b   # TRUE (no more supported)
```

R – Vector and Matrix

Vector

```
a <- c(1,2,3) # a vector
b <- c(4,5,6) # a vector

c <- c(a,b)     # vector (1,2,3,4,5,6)
c[2:4]          # vector (2 3 4)
c[2:4] <- 1     # c vector is (1 1 1 1 5 6)
c[2]   <- 2     # c vector is (1 2 1 1 5 6)
c[c(1,3)] <- 8 # c vector is  8 2 8 1 5 6

d <- sort(c)                      # d vector is 1 2 5 6 8 8
d <- sort(d, decreasing = TRUE) # d vector is 8 8 6 5 2 1
```

R – Vector and Matrix

List *(vector-like but object)*

```
vec_1 <- c(1,2,3)
vec_2 <- c("hello","world")

emptyList <- list()
aList     <- list(vec_1, vec_2)
namedList <- list(first = vec_1, second=vec_2)

length(emptyList) # 0
length(aList)     # 2
length(namedList) # 2

namedList[1]       # a list containing the first object: 1 2 3
namedList[[1]]     # the first object: 1 2 3
namedList[[1][1]   # the first value of the first object: 1
namedList$first    # the first object: 1 2 3
namedList$first[1] # the first value of the first object: 1

append(aList, 42)  # extends a new list (fourth object): 4

merged <- c(aList,namedList) # Merge lists into a new one
```

R – Vector and Matrix

Array *(data storage object for a single type of data, ndim ≥ 1)*

```
# Generate data from 2 to 13 and arranges them
# in two matrices of shape 2x3
array(2:13, dim = c(2, 3, 2))

rownames <- list('a','b')
colnames <- list('c','d','e')
dimnames <- list(rownames, colnames)
a <- array(1:6, dim = c(2, 3), dimnames=dimnames)

dimnames(a) # [["a" "b"] ["c" "d" "e"]]
rownames(a) # "a" "b"
colnames(a) # "c" "d" "e"
dim(a)      # 2 3
nrow(a)     # 2
ncol(a)     # 3
length(a)   # 6
sum(a)      # 21
prod(a)     # 720
```

R – Vector and Matrix

Matrix *(table-like data storage object nrows × ncols)*

```
# Matrix [[1 3 5] [2 4 6]]
matrix(1:6, nrow=2, ncol=3, dimnames=dimnames)

# Matrix [[1 2 3] [4 5 6]]
matrix(1:6, nrow=2, ncol=3, byrow=TRUE, dimnames=dimnames)

# Matrix [[1 1 1] [1 1 1]]
matrix(1, 2, 3)

# Matrix [[1 3] [2 4]]
matrix(c(1, 2, 3, 4),2,2)

# Matrix [[1 0 0] [0 1 0]]
diag(1,2,3)
# Identity Matrix [[1 0 0] [0 1 0] [0 0 1]]
a <- diag(1,3,3)

# array functions are also availale

# [[1 0] [0 1] [0 0]]
a[,1:2] # similar index, dims separated by comma
```

R – Vector and Matrix

DataFrame *(table data storage object with names)*

```r
serials     <- c(218645L, 209259L, 209161L)
firstNames  <- c("Axel", "Matteo", "Théo")
df <- data.frame(first_col = serials, second_col = firstNames)

rowNames <- c("MacGuy","Discreet","HotPC")
df       <- data.frame(df, row.names = rowNames)

print(summary(df) # show sample and metrics

# Extracting friend_name column
who <- df$second_col # is a list

df[1]   # is fist whole column (serials)
who[1]  # gives "Axel"
df[1,2] # gives "Axel"

df["HotPC",] # gives a dataframe for Théo
df[3,1:2]    # gives the same

# matrix functions are also availale
```

R – Control Flow

If-Else

```
if(logical expression){
    statements
}else if(logical expression){
    statements
}else{
    statements
}

# Ternary is one line if-else
x <- if(42 < 1337) "Yes" else "No"

# But R loves vectors... ifelse vector function
x <- ifelse(42 < 1337, "Yes", "No")
```

R – Control Flow

`Repeat` loop

You will be fired if I saw a repeat statement

(because requires `break` insteaf of `repeat-until` or `repeat-while`)

```
repeat {
    statements
    ...
    ...
    if(expression) {
        break
    }
    ...
    ...
    statements
}
```

R – Control Flow

`Foreach` loop

(`For` loop does not exist)

```
for(i in 1:10){
    statements
}

# Is the same
for(i in c(1,2,3,4,5,6,7,8,9,10)){
    statements
}

# Can use for in on vector, array, matrix, dataframe, objects
```

R – Control Flow

Switch case

```r
# switch case is for vectors (function)
# No break, cannot combine cases, always returns

test <- 2
x <- switch(
        test,      # logical expression, the test
        "Newbie", # case 1
        "Noob",   # case 2
        "Naab",   # case 3
        "PGM",    # case 4
) # x is "Noob"


noobAtFPS <- switch(
    x,
    "Noob"="LittleS4T4N",
    "PGM"="Belegkarnil",
    "Newbie"="Sl33per",
    "Naab"="Krogoh"
) # noobAtFPS is "LittleS4T4N"
```

R – Function

Custom functions

```
# Simple function
mysum <- function(x, y){
    x + y                    # last statement is the return
}

# Simple function with a default parameter
myincr <- function(x, incr=1){
    return(x + incr)     # can also specify and force return
}

# Neested functions
generator.power <- function(pow){
    mypow_func <- function(base){
        return(base^pow)
    }
    return(mypow_func)
}
# Create a function of power 2
power.two = generator.power(2)
# Compute 64^2
power.two(64) # is 4096
```

R – Function

Useful functions

```
x <- readline("Please enter an integer:")
x <- as.integer(x)

# read a vector of values
x <- scan(what = integer()) # default what is double()

cat("my test",x)            # only outputs parameters
x <- paste("my test",x)     # Concatenate
res <- print(x)             # also returns the output

x <- "All your bases are belong to us"
nchar(x)          # is 31
substring(x,1,3) # "All"
substr(x,1,3)     # short vesion for lazy man, also "All"
```

Content

1. Introduction
2. R
   - Types and Variables
   - Operators
   - Vector and Matrix
   - Control Flow
   - Function
3. Describing Data
4. Distribution
   - Continuous
   - Discrete
   - Vizualisation
5. Testing Hypotheses
   - Adequacy
   - Conformity
6. Exercise

Describing Data

Load a Dataframe[1]

```
library(tidyverse)
tourism <- read_csv("D:/serie_r/tourismus/tourism.csv")
summary(tourism)
```

```
##       guest        accommodation        stay
##  Min.   :  1.00   Min.   :1.000   Min.   : 1
##  1st Qu.: 38.25   1st Qu.:3.000   1st Qu.: 5
##  Median : 75.50   Median :3.000   Median : 6
##  Mean   : 75.50   Mean   :2.987   Mean   : 6
##  3rd Qu.:112.75   3rd Qu.:3.000   3rd Qu.: 7
##  Max.   :150.00   Max.   :5.000   Max.   :13
##   waitingtime        safety          quality
```

[1] https://www.rdocumentation.org/packages/base/versions/3.6.2/topics/data.frame
[2] https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/median
[3] https://www.rdocumentation.org/packages/base/versions/3.6.2/topics/mean
[4]

Describing Data

`table(•)` counts all occurences of distinct values

```
table(tourism$country)
```

```
#  1  2  3  4
# 64 48 26 12
```

`median(•)`[2] is the value at half of the data.

(separate 50% of higher values from 50% of lower values)

```
median(tourism$safety)
```

```
# [1] 79.5
```

`mean(•)`[3] is the arithmetic mean (average)

```
median(tourism$safety)
```

```
# [1] 79.72
```

[1] https://www.rdocumentation.org/packages/base/versions/3.6.2/topics/data.frame
[2] https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/median
[3] https://www.rdocumentation.org/packages/base/versions/3.6.2/topics/mean
[4] https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/cor

Describing Data

sd(•) var(•) are for normal distribution (later)

```
sd(tourism$safety)
var(tourism$safety)
```

```
# [1] 13.12773
# [1] 172.3372
```

cor(•)[4] computes how features are related.

```
cor(tourism)
```

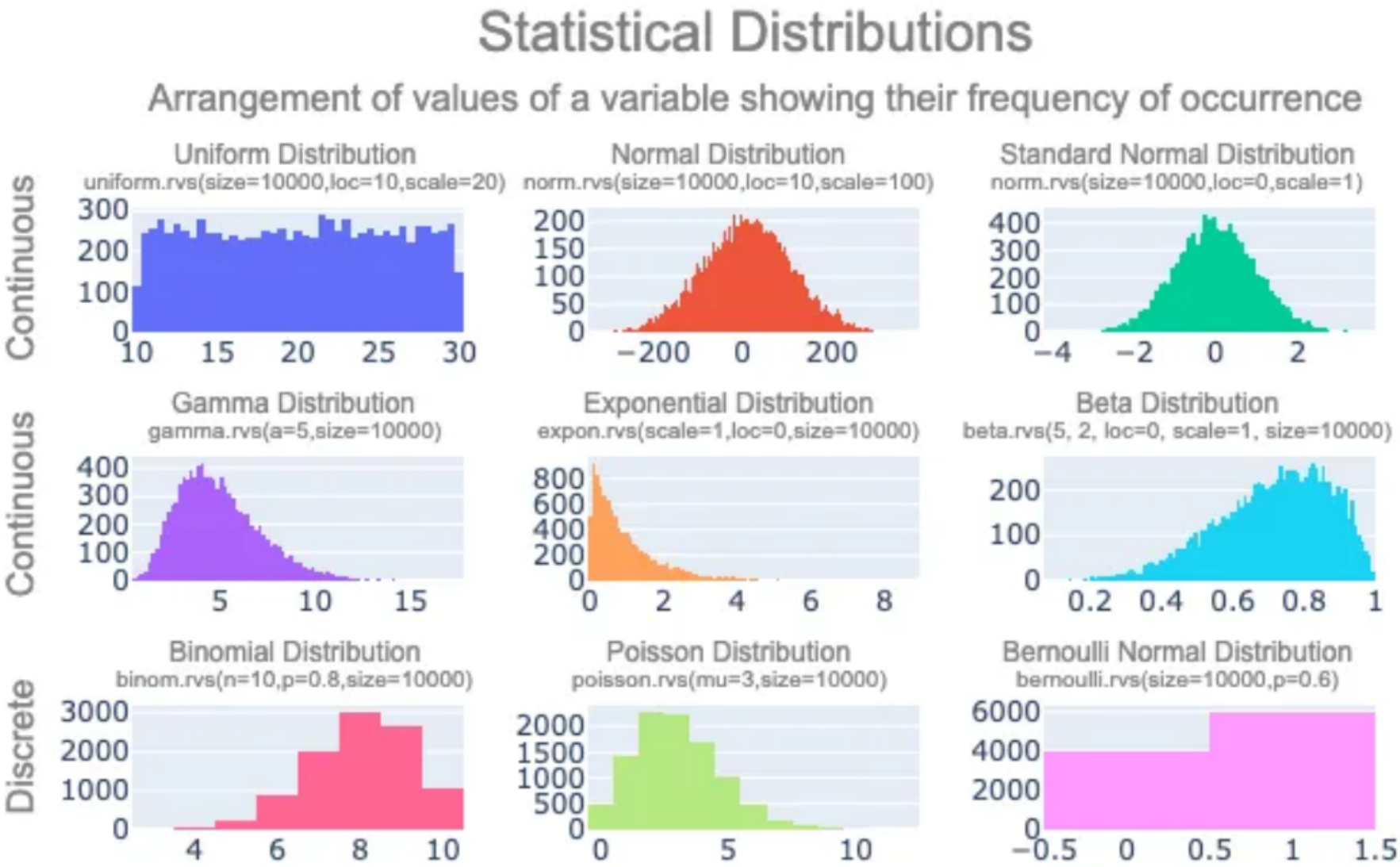
```
#               quality      safety    diversity   waitingtime satisfaction
# quality      1.000000000  0.02820157  0.11103043  0.08553185  0.007709465
# safety       0.028201573  1.00000000 -0.09081689  0.01849999 -0.022505599
# diversity    0.111030428 -0.09081689  1.00000000  0.76150583  0.613930271
# waitingtime  0.085531849  0.01849999  0.76150583  1.00000000  0.793096420
# satisfaction 0.007709465 -0.02250560  0.61393027  0.79309642  1.000000000
```

[1] https://www.rdocumentation.org/packages/base/versions/3.6.2/topics/data.frame
[2] https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/median
[3] https://www.rdocumentation.org/packages/base/versions/3.6.2/topics/mean
[4] https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/cor

Content

1. Introduction
2. R
   - Types and Variables
   - Operators
   - Vector and Matrix
   - Control Flow
   - Function
3. Describing Data
4. Distribution
   - Continuous
   - Discrete
   - Vizualisation
5. Testing Hypotheses
   - Adequacy
   - Conformity
6. Exercise

Distribution



Distribution

Continuous Uniform

$$x \sim \mathcal{U}_{[a,b]} \equiv f(x) = \begin{cases} \frac{1}{b-a} & \text{If } x \in [a,b] \\ 0 & \text{Otherwise} \end{cases}$$

where

$\frac{1}{2}(a+b)$ is the mean (expected value)

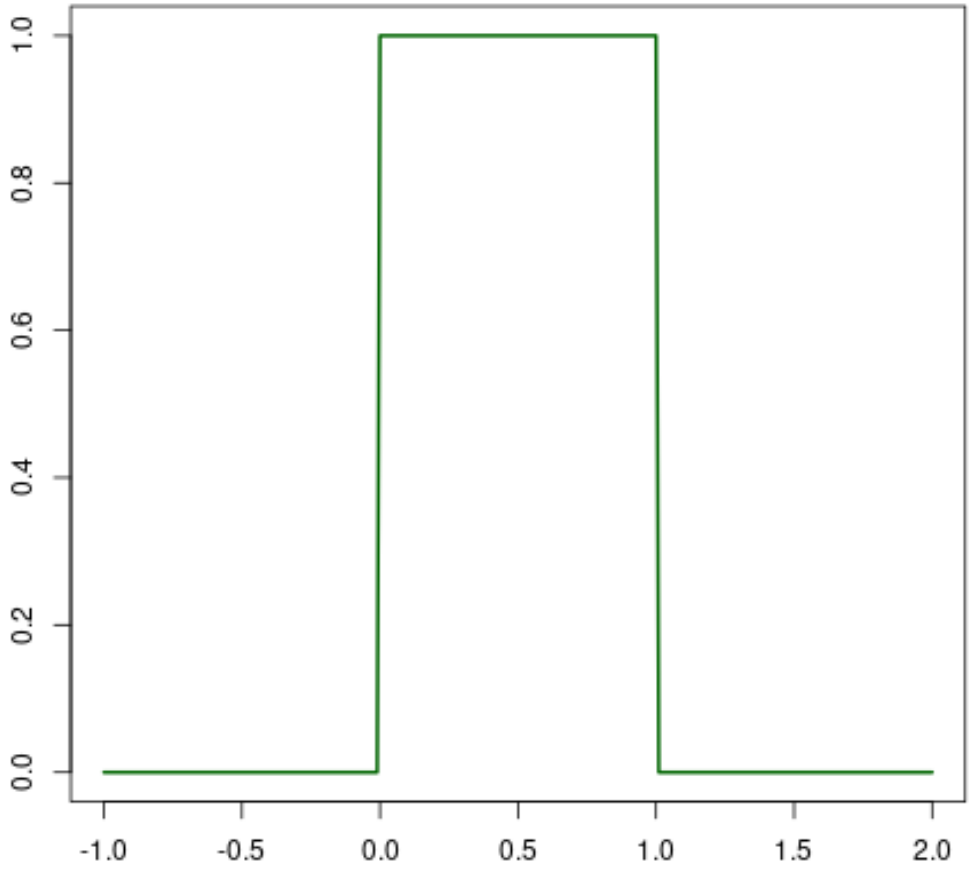
$\sqrt{\frac{1}{12}(b-a)^2}$ is the standard deviation[5]

$\frac{1}{12}(b-a)^2$ is the variance (dispersion)

[5] In french: *écart type*

Distribution

Continuous Uniform




```
x      <- seq(-1, 2, 0.01)
unif <- dunif(x, min = 0, max = 1)
plot(x, unif , type = "l",  ylim = c(0, 1),
        ylab = "", lwd = 2, col = "darkgreen")
```

Distribution

Continuous – Normal/Gaussian

$$x \sim \mathcal{N}(\mu, \sigma) \equiv f(x) = \frac{1}{\sigma\sqrt{2\pi}}\, e^{\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

where

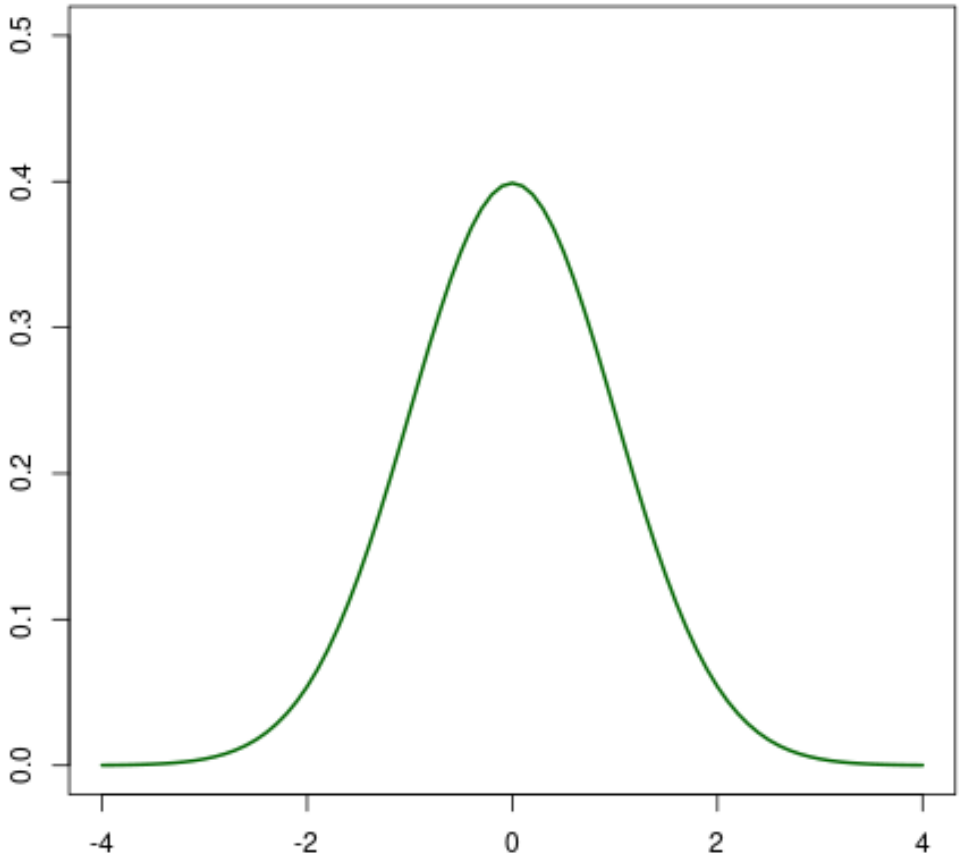
μ is the mean (expected value)

σ is the standard deviation

σ^2 is the variance (dispersion)

Distribution

Continuous – Normal/Gaussian




```
x      <- seq(-4, 4, 0.1)
normal <- dnorm(x, mean = 0, sd = 1)
plot(x, normal , type = "l",  ylim = c(0, 0.5),
         ylab = "", lwd = 2, col = "darkgreen")
```

Distribution

Discrete – Binomial

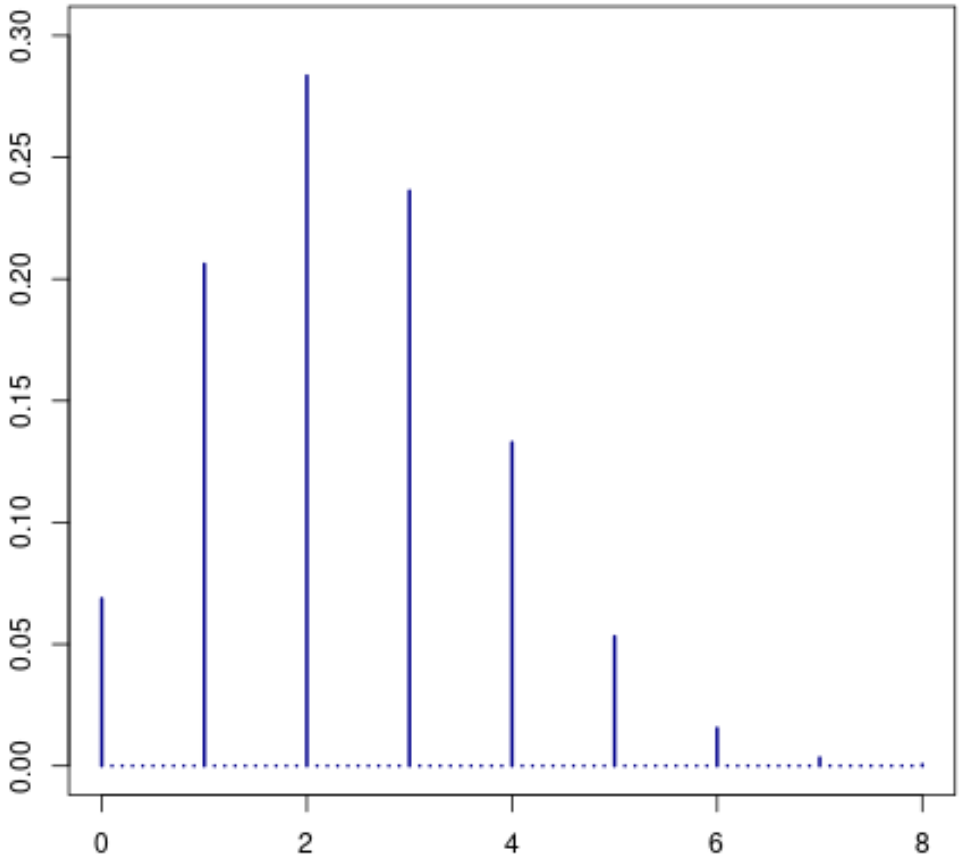
$$k \sim \mathcal{B}(n,p) \equiv f(k) = \left(\begin{array}{c} n \\ k \end{array} \right) p^k (1-p)^{n-k} = \frac{n!}{k!(n-k)!} p^k (1-p)^{n-k}$$

where

np	is the mean (expected value)
$\sqrt{np(1-p)}$	is the standard deviation
$np(1-p)$	is the variance (dispersion)

Distribution

Discrete – Binomial




```
x      <- seq(0, 8, 0.1)
binom <- dbinom(x, size=12, prob=0.2)
plot(x, binom , type = "h", ylim = c(0, 0.3),
        ylab = "", lwd = 2, col = "darkblue")
```

Distribution

Discrete – Poisson

$$k \sim \mathrm{Pois}(\lambda) \equiv f(k) = \frac{\lambda^k \; e^{-k}}{k!}$$

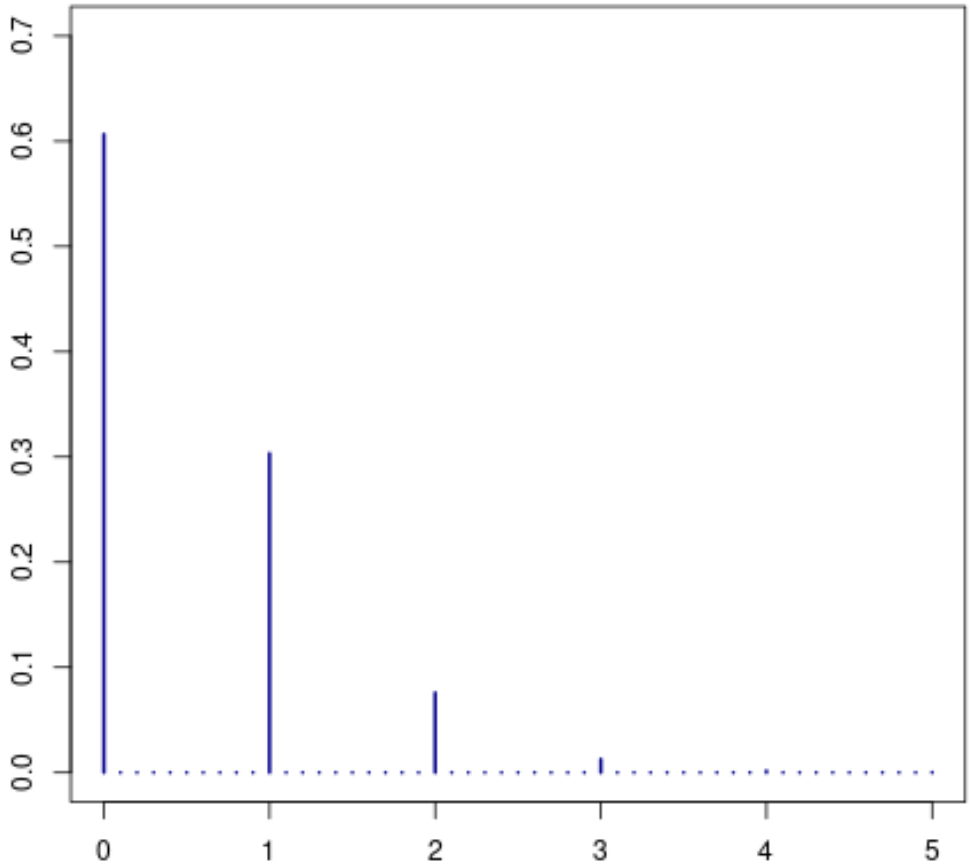
where

$\sqrt{\lambda}$ is the standard deviation

λ^2 is the variance (dispersion) **and** the mean

Distribution

Discrete – Poisson



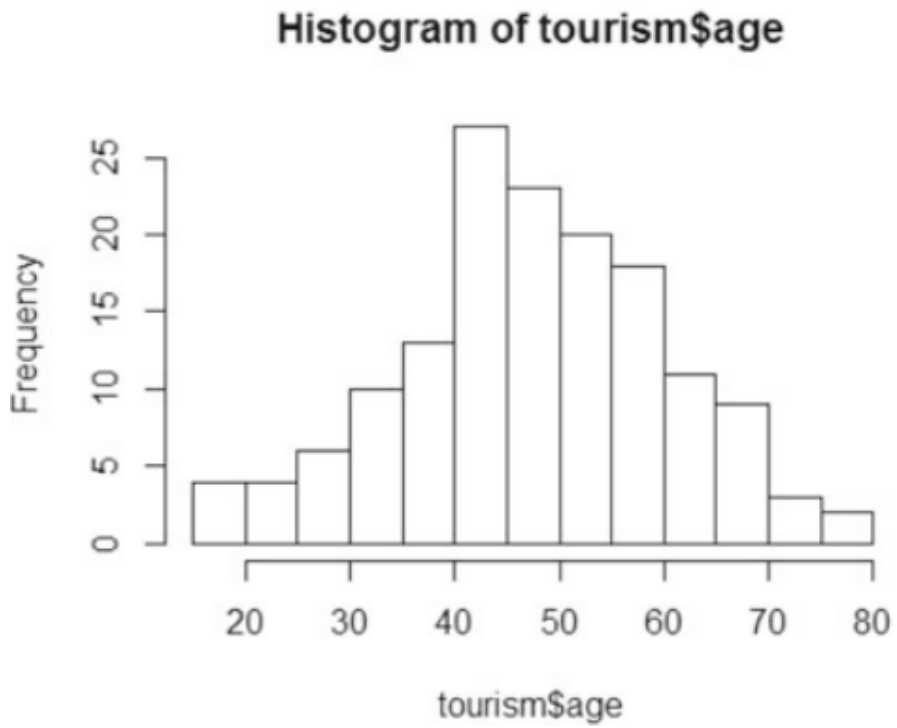

```
x      <- seq(0, 5, 0.1)
pois   <- dpois(x, lambda=0.5)
plot(x, pois , type = "h",  ylim = c(0, 0.7),
        ylab = "", lwd = 2, col = "darkblue")
```

Distribution

Vizualisation

Graphical/Vizualisation Testing

```
library(tidyverse)
tourism <- read_csv("D:/serie_r/tourismus/tourism.csv")
hist(tourism$age)
```



Content

1. Introduction
2. R
   - Types and Variables
   - Operators
   - Vector and Matrix
   - Control Flow
   - Function
3. Describing Data
4. Distribution
   - Continuous
   - Discrete
   - Vizualisation
5. Testing Hypotheses
   - Adequacy
   - Conformity
6. Exercise

Testing Hypotheses

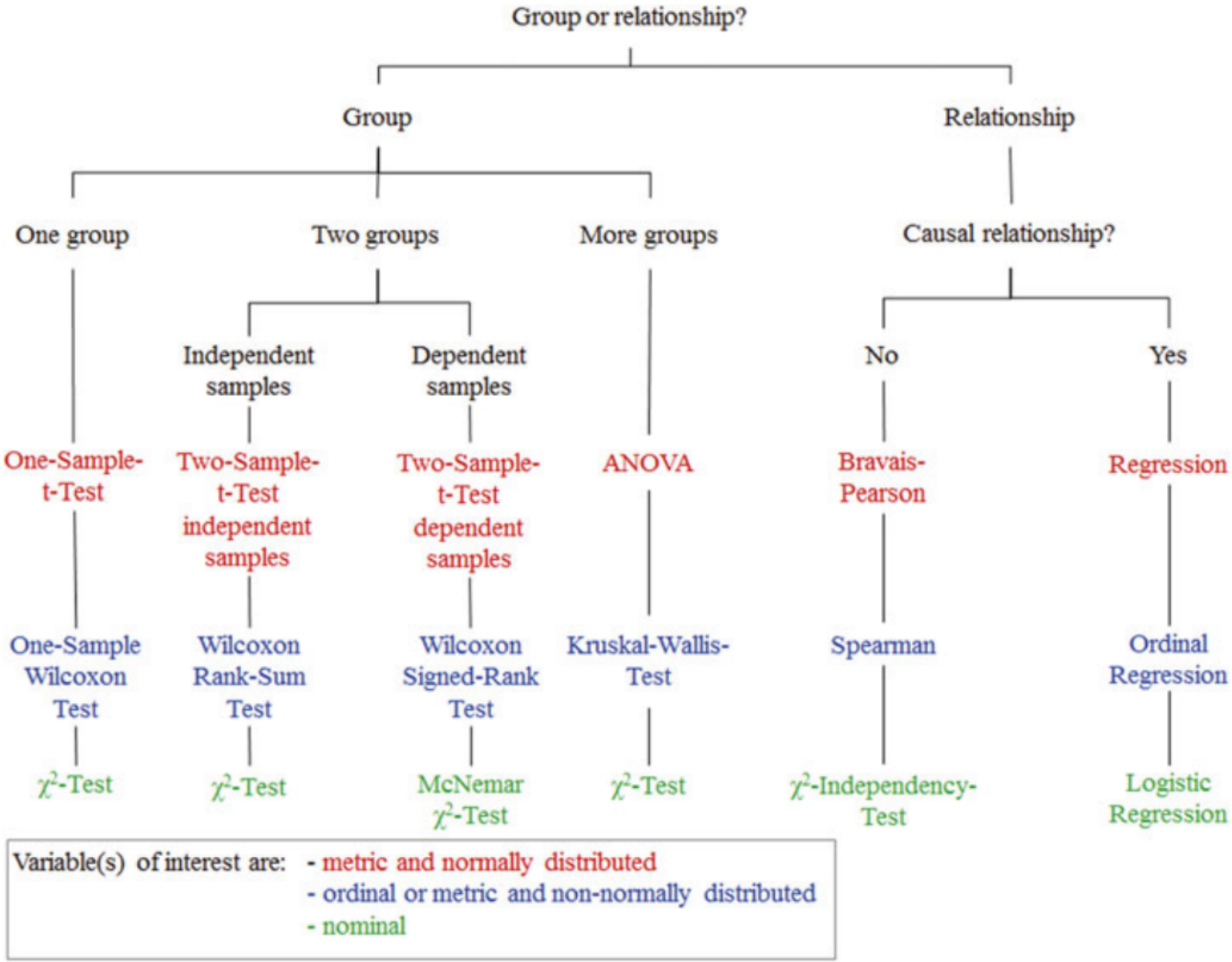
Adequacy testing

is a **probability test** performed on the estimators of **two random variables** under the assumption that they follow the **same distribution** (*probability law*).

Conformity testing

is a **probability test** performed on the estimators of **two random variables** following **the same distribution**, under the assumption that their **parameters are identical**.

Testing Hypotheses



Testing Hypotheses

Adequacy – Kolmogorov-Smirnov (continuous)

Let $\bar{A} \sim \mathcal{N}_A(\mu_A, \sigma_A^2)$ and the Kolmogorov-Smirnov test is written $KS(\bar{A}, \mathcal{N}_A)$.

$H_0 \equiv \mathcal{N}_A(\mu_A, \sigma_A^2) = F(x) \sim \mathcal{N}(\mu, \sigma^2)$ (null-hypothesis, what we test)

Consider the empirical distribution function

$$F_n(x) = \frac{1}{n}\sum_{i=1}^{n} d(x_i, x), \qquad d(x_i, x) = \begin{cases} 1 & \text{If } x_i \leq x \\ 0 & \text{Otherwise} \end{cases}$$

Use $D(x) = \max[\max_x(F_n(x) - F(x)), \max_x(F(x) - F_n(x))]$

H_0 can be reformulated as *the maximum difference between the empirical function and the distribution must be small*.

In R: `ks.test` (return a p-value which is $\mathbb{P}(x|H_0)$)
In Julia: `HypothesisTests.ExactOneSampleKSTest`

Reminder: A small p value (<=0.05), you should reject the null hypothesis.

Testing Hypotheses

Adequacy – Pearson χ^2 (discrete)

Let $\bar{A} \sim \mathcal{N}_A(\mu_A, \sigma_A^2)$, $a_i \in \mathbb{A} = \{0, \ldots, j\}$ and the Pearson χ^2 test is written $\chi^2(\bar{A}, \mathcal{N}_A)$.

$H_0 \equiv \mathcal{N}_A(\mu_A, \sigma_A^2) \sim \mathcal{N}(\mu, \sigma^2)$

$$\chi^2 = m \sum_{i=1}^{m} p_i \left(\frac{\frac{O(i)}{m} - p_i}{p_i} \right)^2, \qquad O(i) = \sum_{j}^{n} \begin{cases} 1 & \text{If } a_j = i \\ 0 & \text{Otherwise} \end{cases}$$

p_i is the theoretical frequency (area (i.e. integral) under the Gaussian distribution), while $O(i)$ is the observed frequency.

H_0 can be reformulated as $H_0 \equiv \mathbb{P}(\chi^2 < \chi_\alpha^2)$ where α is a critical value (typically 0.05).

In R: `chisq.test`

In Julia: `HypothesisTests.ChisqTest`

Testing Hypotheses

Conformity – Ficher-Snedecor F-test (testing variances)

Let $\bar{A} \sim \mathcal{N}_A(\mu_A, \sigma_A^2)$ be a n-sample and $\bar{B} \sim \mathcal{N}_B(\mu_B, \sigma_B^2)$ be a m-sample.

$H_0 \equiv \sigma_A = \sigma_B, H_1 \equiv \sigma_A \neq \sigma_B$

If $\bar{A}, \bar{B}$ follow a same law, we can write $\bar{C} = \bar{A} \cup \bar{B}$ is a $(n+m)$-sample. With the maximum likelihood[5] method:

$$\hat{\sigma}^2 = \frac{1}{n+m}\left(\sum_{i=0}^{n}(a_i - \mu_A)^2 + \sum_{i=0}^{m}(b_i - \mu_B)^2\right)$$

Which will be used as an estimator if $\mathcal{N}_A \equiv \mathcal{N}_B$.

In R: `var.test`

In Julia: `HypothesisTests.VarianceFTest`

[5] Maximum de vraissemblance

Testing Hypotheses

Conformity – Ficher-Snedecor F-test (testing variances)

Let $\bar{A} \sim \mathcal{N}_A(\mu_A, \sigma_A^2)$ be a n-sample and $\bar{B} \sim \mathcal{N}_B(\mu_B, \sigma_B^2)$ be a m-sample.

$H_0 \equiv \sigma_A = \sigma_B, H_1 \equiv \sigma_A \neq \sigma_B$

Taking the quotient of the likelihood functions:

$$\frac{L_A}{L_B} \qquad \left(\frac{\frac{n-1}{n}\hat{\sigma}_A}{\frac{m-1}{m}\hat{\sigma}_B} \right)$$

$$F = \quad \frac{n}{n-1}\frac{m-1}{m}\left(\frac{L_A}{L_B}\right)^{\frac{2}{n}} \quad \frac{\hat{\sigma}_A^2}{\hat{\sigma}_B^2}$$

$F(n, m)$ is a random variable that follows a χ^2 distribution/law.

In R: `var.test`

In Julia: `HypothesisTests.VarianceFTest`

Testing Hypotheses

Conformity – Student t-test (testing means)

Let $\bar{A} \sim \mathcal{N}_A(\mu_A, \sigma_A^2)$ be a n-sample and $\bar{B} \sim \mathcal{N}_B(\mu_B, \sigma_B^2)$ be a m-sample.

$H_0 \equiv \mu_A = \mu_B, H_1 \equiv \mu_A \neq \mu_B$

Similar to the F-test:

$$t = \frac{L_A}{L_B}^{\frac{2}{m+n}} = \left(\frac{\sum_{i=1}^{n} \left(a_i - \frac{n\hat{\mu_A} + m\hat{\mu_B}}{n+m} \right)^2 + \sum_{i=1}^{m} \left(b_i - \frac{n\hat{\mu_A} + m\hat{\mu_B}}{n+m} \right)^2}{\sum_{i=1}^{n} (a_i - \hat{\mu}_A)^2 + \sum_{i=1}^{m} (b_i - \hat{\mu}_B)^2} \right)$$

$t(n, m)$ is a random variable that follows a χ^2 distribution/law.

In R: `t.test`

In Julia: `HypothesisTests.OneSampleTTest`

Content

1. Introduction
2. R
   - Types and Variables
   - Operators
   - Vector and Matrix
   - Control Flow
   - Function
3. Describing Data
4. Distribution
   - Continuous
   - Discrete
   - Vizualisation
5. Testing Hypotheses
   - Adequacy
   - Conformity
6. Exercise

Exercise

Download the file `algos.csv` from eCampus.

Statement

You're in charge of a team of 3 developers (Mr. Cremer, Mr. Quévy and Mr. Lerat), and each of them comes up with a different algorithm for solving the same problem. You measured the execution time:

$$\mathrm{time}(\text{algorithm}_i(\text{input})) = a \times \text{input}^k$$

Your random variable X is the value of k.

Based on the random variable $X = k$:

1. Is one algorithm better than another?
2. Which would you use?
3. Which would you use on an increasing number of inputs?

Exercise

Based on the random variable $X = k$:

1. Is one algorithm better than another?
2. Which would you use?
3. Which would you use on an increasing number of inputs?

Hints :

1. Draw a scatter plot. What do you observe?
2. Draw a histogram by partitioning the data. What do you observe?
3. Estimate the parameters of a normal distribution for each algorithm.
4. Do the observations looks like a normal distribution?
5. Are the algorithms identical?